# *LecAssist: An AI Based Agent System for Recording, Summarising and Managing University Lectures*

## *FYP Code: 23F-64*

## *Software Design Specification*

BY

ASAD SALEEM, CMS. ID: 023-23-0213

SHOAIB AHMED, CMS. ID: 023-23-0231

ASGHAR ALI, CMS. ID: 023-23-0218




Supervised by: Dr. Saif Hassan _____

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY

SUKKUR IBA UNIVERSITY

NOVEMBER 2026

*1. INTRODUCTION* ..... ***ERROR! BOOKMARK NOT DEFINED.***

1.1. Purpose of Document ..... **Error! Bookmark not defined.**
1.2. Definitions, acronyms, and abbreviations ..... **Error! Bookmark not defined.**

*2. OVERALL SYSTEM DESCRIPTION* ..... ***ERROR! BOOKMARK NOT DEFINED.***

2.1. Project Background ..... **Error! Bookmark not defined.**
2.2. Project Objectives ..... **Error! Bookmark not defined.**

*3. SYSTEM ARCHITECTURE* ..... ***ERROR! BOOKMARK NOT DEFINED.***

*4. DOMAIN MODEL* ..... ***ERROR! BOOKMARK NOT DEFINED.***

*5. CLASS DIAGRAM* ..... ***ERROR! BOOKMARK NOT DEFINED.***

*6. DATABASE DIAGRAM* ..... ***ERROR! BOOKMARK NOT DEFINED.***

*7. ENTITY RELATIONSHIP DIAGRAM (ERD)* ..... ***ERROR! BOOKMARK NOT DEFINED.***

*8. SEQUENCE DIAGRAM* ..... ***ERROR! BOOKMARK NOT DEFINED.***

*9. SYSTEM INTERFACE DESIGN* ..... ***ERROR! BOOKMARK NOT DEFINED.***

*10. TEST CASES* ..... ***ERROR! BOOKMARK NOT DEFINED.***

# Introduction

## Purpose of Document

This document describes how LecAssist is built. The Software Requirements Specification states what the system must do; this document states how those requirements are turned into software. It gives the architecture of the application, the domain model, the classes, the database tables and their relationships, the order of messages between the parts of the system, the design of the screens, and the test cases that will be used to check the system. The design follows the layered plan given in the proposal. Every layer is built and tested before the next one is started, because a wrong transcript gives a wrong summary and a wrong summary gives a wrong assignment. The parts marked as already implemented in this document are in the project repository at https://github.com/asadsehto/lecassist.

## Definitions, acronyms, and abbreviations

| Term | Meaning |
|---|---|
| ASR | Automatic Speech Recognition, the conversion of speech into text. |
| WER | Word Error Rate, the standard measure of transcription accuracy. |
| Diarization | Separating a recording into turns and labelling which speaker spoke each turn. |
| Code switching | Changing language inside one sentence, for example Urdu and English in the same line. |
| LLM | Large Language Model, the model that writes the summary and drives the agents. |
| Agent | A language model that is given tools and performs an action, not only text. |
| Tool calling | The mechanism by which the model asks the program to run a named function with named arguments. |
| RAG | Retrieval Augmented Generation, answering a question from stored documents. |
| Moodle | The eLearning portal and LMS used by Sukkur IBA University at elearning.iba-suk.edu.pk. |
| Web service token | The key issued by the Moodle administrator that allows the REST API to be called for a teacher account. |
| Ollama | A tool that runs language models locally on the teacher's own machine. |
| OpenClaw | An open source, self hosted agent gateway that connects one agent to messaging channels such as Telegram and WhatsApp. |
| Playwright | A browser automation library used to operate the Moodle pages that have no web service. |
| FastAPI | The Python framework used for the local service inside the desktop application. |
| SQLite | The local file based database used for lectures, transcripts and summaries. |
| Vector index | A store of text embeddings (Chroma or FAISS) used for meaning based search. |
| Embedding | A list of numbers that represents the meaning of a piece of text, used for search. |
| PK / FK | Primary key and foreign key in the database tables. |

# Overall System Description

## Project Background

A lecture does not end when the professor leaves the room. Notes still have to be written for the students who were absent, the assignment announced out loud has to be recorded and its deadline tracked, supporting material has to be placed on the eLearning portal, and marks eventually have to be entered into the LMS. The work is repetitive and it eats hours that a teacher would rather spend teaching.

Transcription on its own is already a solved problem and is not the contribution of this project. Existing tools such as Otter.ai, Fireflies and the meeting summaries inside Zoom and Teams stop once they have produced a file. They also go weak on the mixed Urdu and English speech that is normal in a Pakistani classroom, they treat every recording as a separate event, they deliver to email rather than to WhatsApp, and they require classroom audio to be uploaded to another company's server.

LecAssist is a desktop application that turns one recording into every document and action that follows from it. The teacher presses record once. The system cleans the audio, transcribes mixed Urdu and English speech locally, separates the teacher from the students, writes the summary, pulls out the assignments and deadlines, files the lecture under the correct subject and section, performs the portal and LMS work after approval, delivers the notes to the class group, and keeps the whole semester as a searchable record.

## *Project Objectives*

1. To record and process classroom audio in real conditions, including background noise, pauses, interruptions and files recorded on other devices.
2. To produce accurate transcripts of mixed Urdu and English lecture speech, with the teacher's speech separated from the students', and with every line linked to its position in the audio.
3. To convert a transcript into useful academic output: a summary in more than one style, a lecture title, a list of key terms, important moments, and the assignments announced in class with their deadlines.
4. To organise the archive from the teacher's timetable and to compare the delivered lectures with the uploaded course outline.
5. To perform the office work that follows a lecture on the Moodle portal and the LMS, with approval of the teacher required before any outside action.
6. To deliver the output to the students and to the teacher on Telegram and WhatsApp through a chat session that accepts replies and new instructions.
7. To index the full semester so that questions can be answered from all lectures with references, and so that quizzes and complete exam papers can be generated.
8. To allow the complete system to run offline on the teacher's own machine, and to allow a paid model from Anthropic or OpenAI to be selected instead.

## *System Architecture*

LecAssist is built as one desktop application in five layers. Each layer talks only to the layer below it, so a change in the interface does not touch the audio pipeline and a change of model provider does not touch the agents. The figure below shows the layers and the external systems.

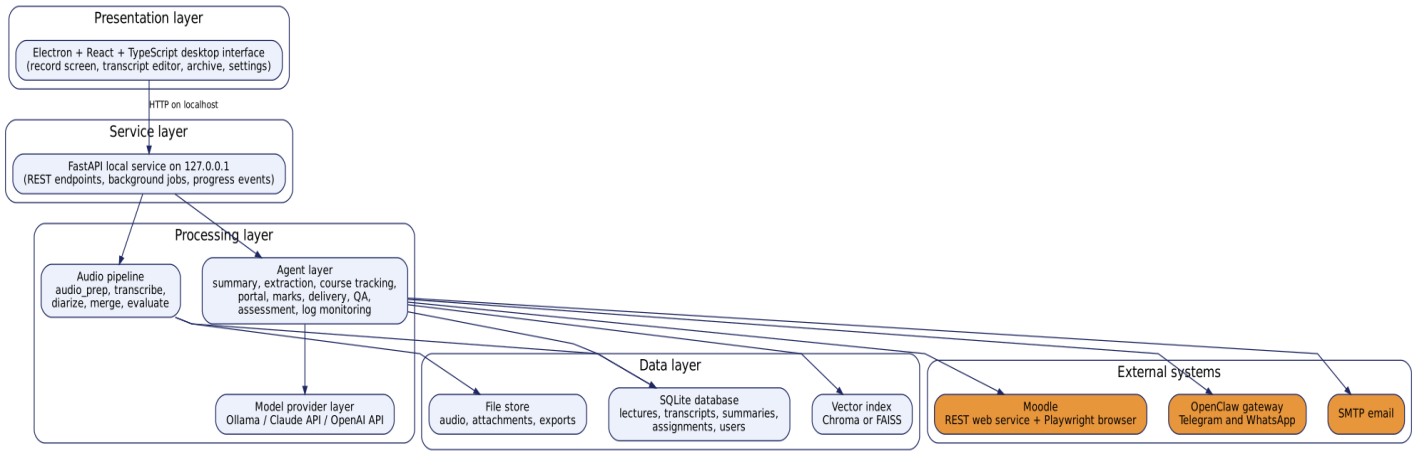



*Figure 1: System architecture of LecAssist*

| Layer | What it contains | Why it is separate |
|---|---|---|
| Presentation | An Electron window running a React and TypeScript interface: the record screen, the transcript editor, the summary view, the archive, the calendar, the approval dialog and the settings page. | The interface holds no logic of its own. It calls the local service and shows what comes back, so the same functions can later be driven from the chat session on the phone. |
| Service | A Python 3.11 FastAPI service bound to 127.0.0.1 only. It exposes REST endpoints for lectures, transcripts, summaries, actions and search, runs long work as background jobs and streams progress back to the interface. | Recording, transcription and agent work take minutes. Keeping them behind a local service keeps the window responsive and lets a job continue after the screen is closed. |
| Processing | The audio pipeline (prepare, transcribe, diarize, merge, evaluate), the model provider layer, and the ten agents of the system. | This is where the project's own work sits. The audio pipeline and the agents never call each other directly; they exchange stored records through the data layer. |
| Data | A SQLite database, a vector index (Chroma or FAISS) and a folder of audio files, attachments and exports, all on the teacher's machine. | One local store keeps the system usable with no internet and makes backup a file copy. |
| External | Moodle through its REST web service and through Playwright browser automation, Telegram and WhatsApp through the OpenClaw gateway, and SMTP for email. | Every outside system is reached from one place, behind the approval step, so a failure outside cannot damage the stored archive. |

Two design decisions carry most of the risk of the project and are therefore fixed here. First, speech recognition always runs locally and the language model is chosen behind one interface, so that the same code path works for a local Ollama model and for a paid Claude or OpenAI API, and the choice is one setting and not a change in design. Second, the portal work is split: everything that can be read is read through the Moodle web service, which is stable, and only the actions that Moodle does not expose as a service are performed by the browser agent, which is the fragile part and is kept behind the approval step.

## *Domain Model*

The domain model is given in two parts, because the teaching structure and the contents of one lecture are easier to read separately. It shows the real things the system deals with and how they are related, before any decision about tables or classes. A teacher teaches subjects, each subject is divided into sections with timetable slots and a course outline, and each section delivers lectures. One lecture carries its recording, its transcript, its summaries, its key terms, its highlights, the assignments announced in it and the material attached to it. Everything the agents do and everything that is delivered points back to a lecture.

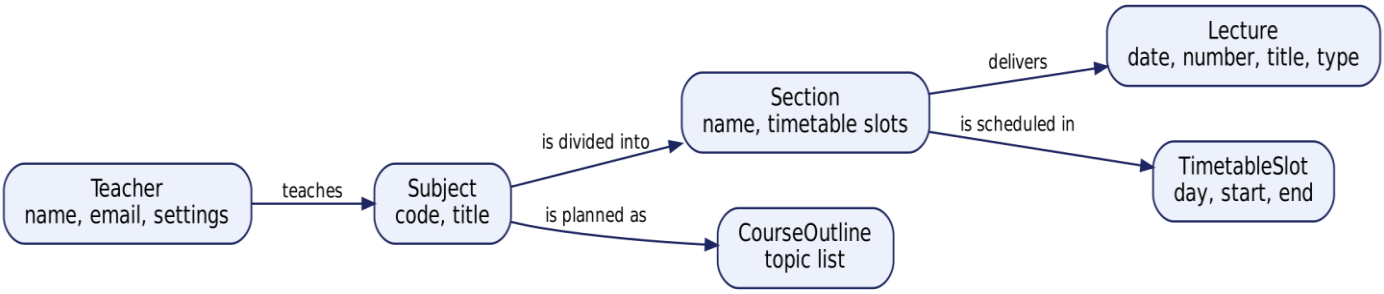



*Figure 2: Domain model, part 1: the teaching structure*

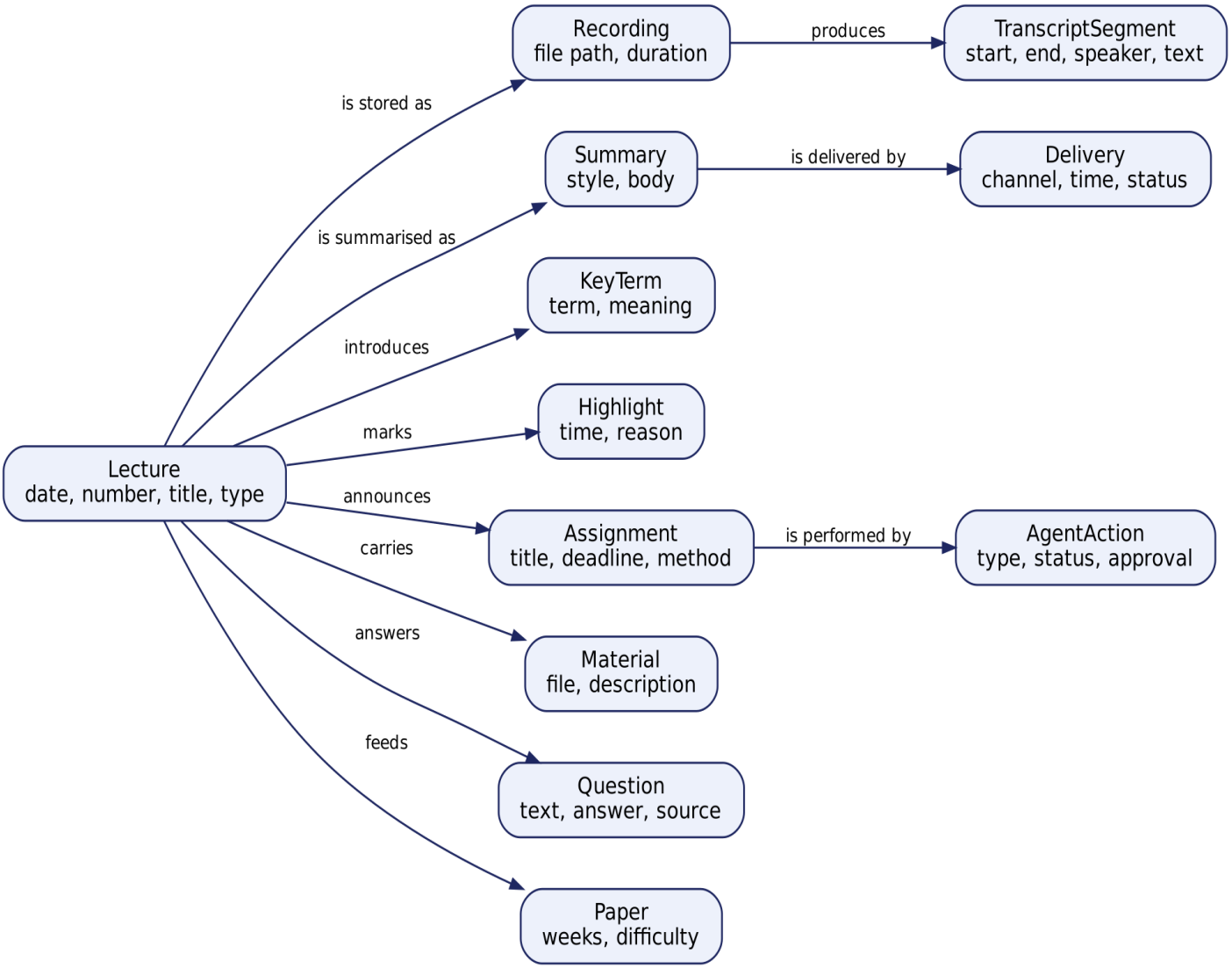



*Figure 3: Domain model, part 2: everything that belongs to one lecture*

The important rule in this model is that the lecture is the centre. A summary belongs to a lecture and not to a subject, an assignment belongs to the lecture in which it was announced, and every answer the system gives in semester search can therefore be traced back to one lecture and one moment in its audio.

## *Class Diagram*

The class diagram is given in two parts. The first part is the audio and transcription chain, which already exists in the repository as modules. The second part is the model provider interface, the agents built on top of it and the repository class that is the only one allowed to touch the database.

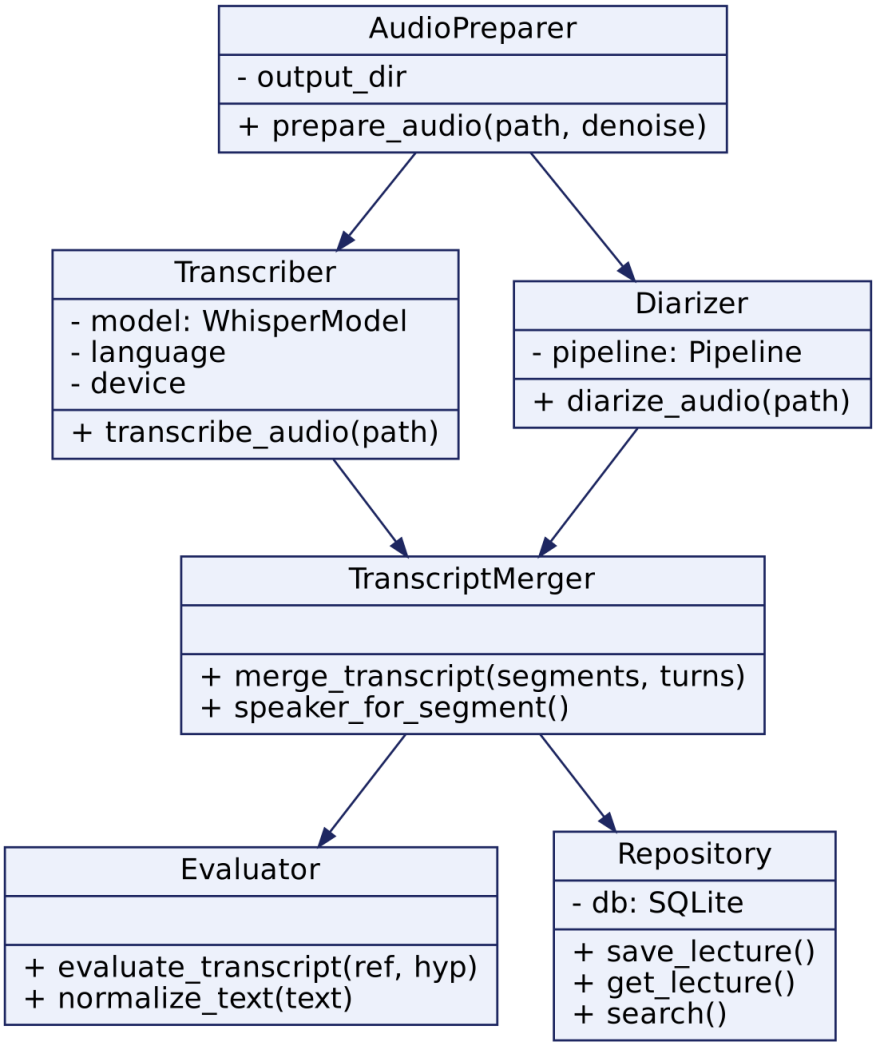



*Figure 4: Class diagram, part 1: the audio and transcription classes*

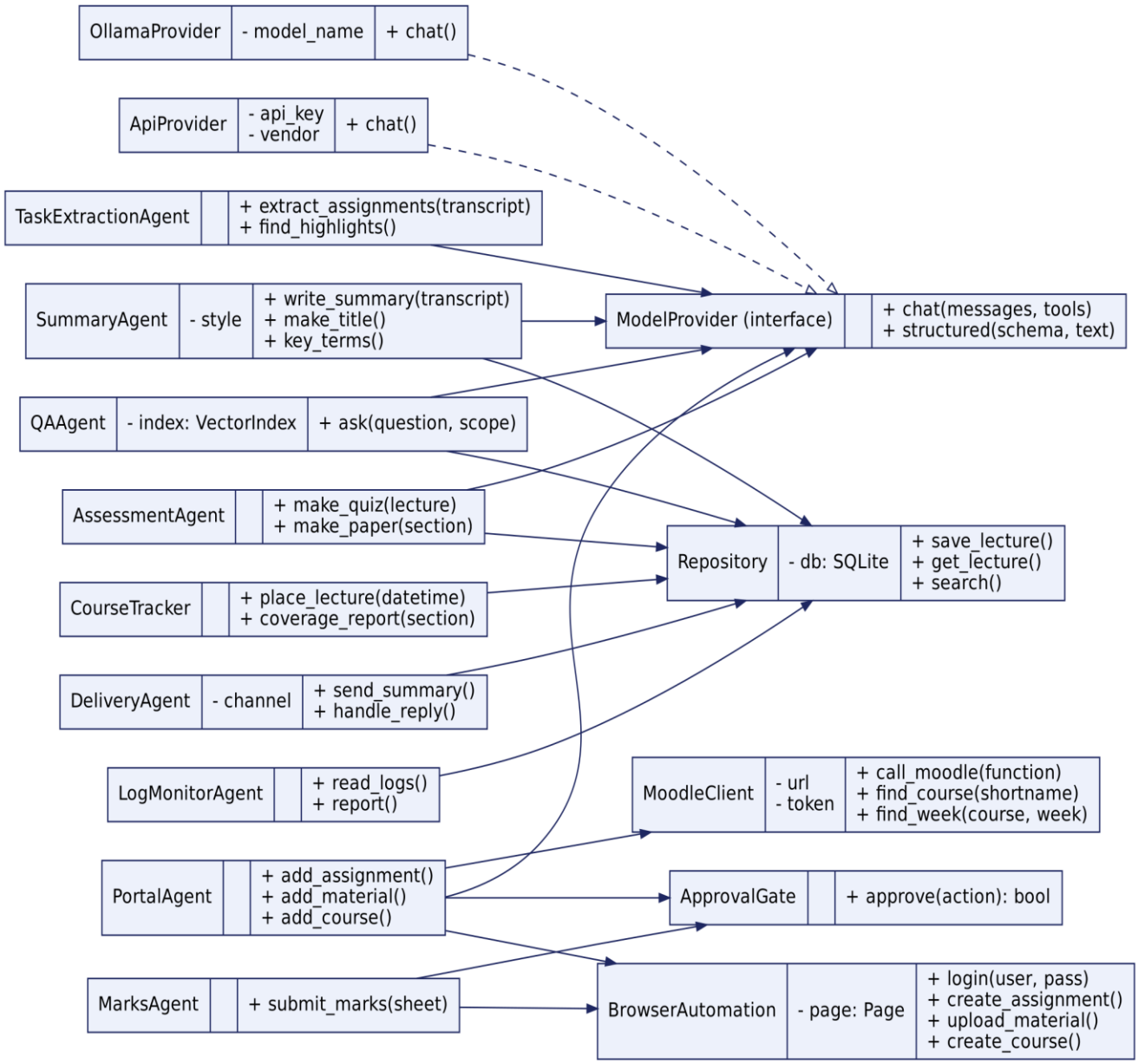



*Figure 5: Class diagram, part 2: the model provider, the agents and the repository*

| Class | Responsibility |
|---|---|
| AudioPreparer | Converts any input file to 16 kHz mono WAV with FFmpeg and optionally writes a noise reduced copy. |
| Transcriber | Runs faster-whisper with Whisper large-v3-turbo on CPU or CUDA with a mixed Urdu and English initial prompt and returns timed segments. |
| Diarizer | Runs pyannote speaker-diarization-3.1 and returns speaker turns. |
| TranscriptMerger | Gives each segment the speaker whose turn overlaps it most and writes the merged transcript as text and JSON. |
| Evaluator | Normalises text and computes word error rate with substitution, deletion and insertion counts against a reference transcript. |
| ModelProvider | One interface for chat, tool calling and schema based output. OllamaProvider and ApiProvider implement it. |
| SummaryAgent | Writes the summary in the chosen style, the lecture title and the key term list. |
| TaskExtractionAgent | Returns announced assignments and highlights as structured records with title, deadline and submission method. |
| CourseTracker | Places a finished lecture under the right subject and section from its date and time, and builds the coverage report against the outline. |
| MoodleClient | Calls the Moodle REST web service to find a course by short name and a week section by number. |
| BrowserAutomation | Drives Chromium through Playwright to log in, create an assignment with a due date and attachment, upload material and create a course. |
| ApprovalGate | Shows the prepared action to the teacher and returns true only when it is approved. |
| PortalAgent / MarksAgent | Combine MoodleClient, BrowserAutomation and ApprovalGate into the tools the language model is allowed to call. |
| DeliveryAgent | Sends the summary through OpenClaw and keeps the chat session, and records delivery status. |
| QAAgent | Answers a question from the vector index within the chosen scope and returns the lecture and date of each source. |
| AssessmentAgent | Generates quizzes and complete papers from stored lectures. |
| LogMonitorAgent | Reads the logs of the other agents and reports failures, repeats, unusual access and cost. |
| Repository | The only class that touches SQLite and the vector index, so that no agent writes to the database directly. |

## *Database Diagram*

All data is stored in one SQLite file on the teacher's machine. The diagram below shows the tables, their main columns and the foreign keys between them. Audio files and attachments are kept on disk and the database stores their paths, because storing large audio inside the database would make backup and search slow.

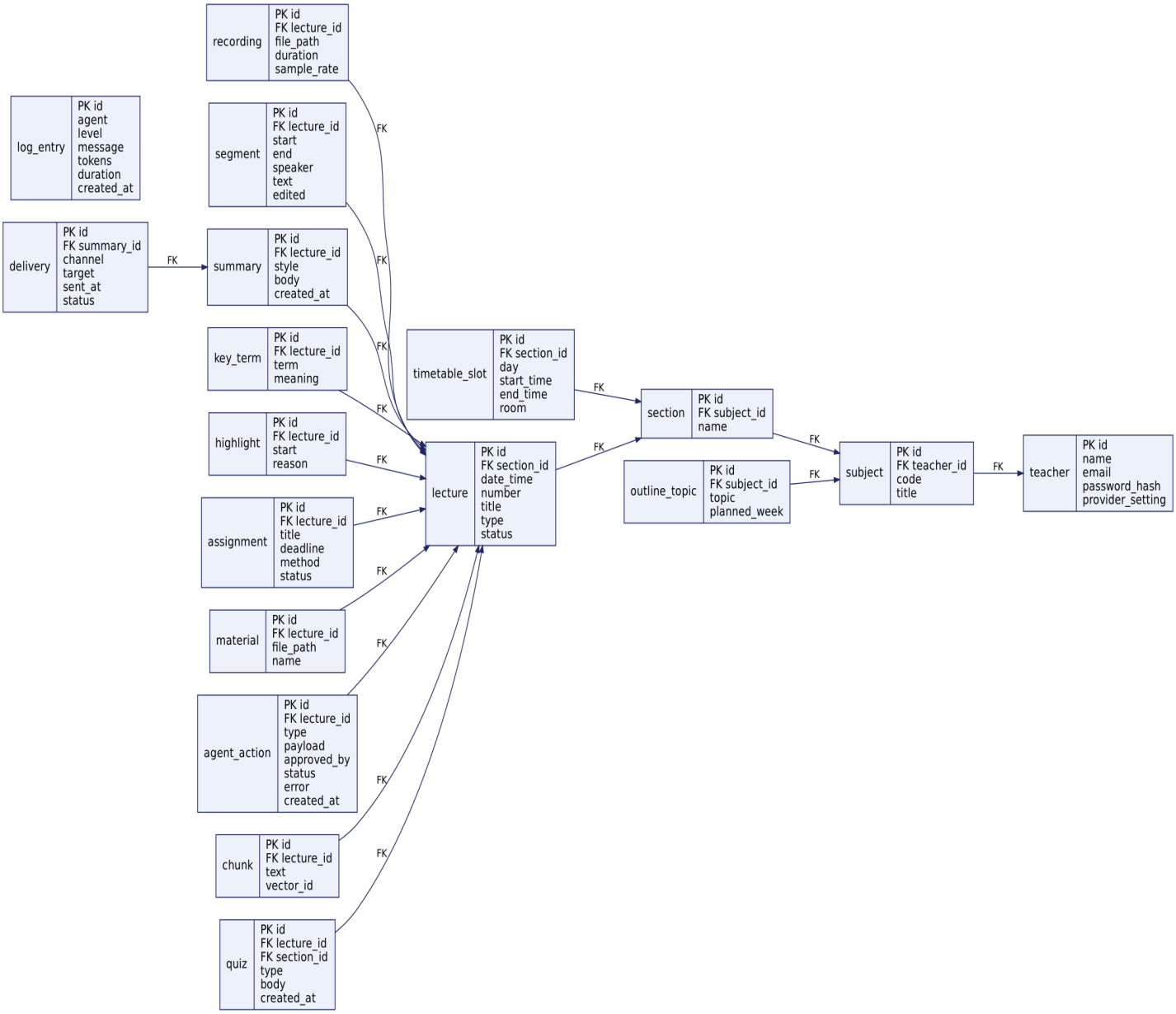



*Figure 6: Database diagram*

| Table | Purpose and key columns |
|---|---|
| teacher | One row per user of the installation. Holds the login, the selected model provider and the settings. Every other table belongs to a teacher through the subject. |
| subject, section, timetable_slot | The teaching structure uploaded once at the start of the term. The slot gives day, start time and end time, which is what decides where a finished recording is filed. |
| outline_topic | The planned topics of a subject, used to build the coverage report for each section. |
| lecture | One row per class. Holds the date and time, the lecture number inside its section, the generated title, the type (regular, extra, revision, guest) and the processing status. |
| recording | The audio of a lecture: file path, duration and sample rate. |
| segment | One row per transcript line: start time, end time, speaker label, text and an edited flag that marks a line the teacher corrected. |
| summary | One row per style generated for a lecture, with the body and the time it was written. |
| key_term, highlight | The glossary of the lecture and the moments the teacher marked as important. |
| assignment | The task found in the speech, with its deadline, submission method and upload status. |
| material | Files attached to the lecture: whiteboard photographs, slide decks and readings. |
| agent_action | Every outside action attempted by an agent, with its type, its parameters, who approved it, the result and the error text if it failed. This is the audit trail of the system. |
| delivery | One row per message sent, with the channel, the target group or account, the time and the status. |
| chunk | The pieces of transcript that are embedded for meaning based search, each pointing to the vector stored in the index. |
| quiz | Generated quizzes and papers, kept with the lecture or the section they were built from. |
| log_entry | Agent logs read by the monitoring agent: level, message, tokens used and duration. |

## *Entity Relationship Diagram (ERD)*

The entity relationship diagram shows the same data as the database diagram, written as entities and the relationships between them with their cardinality.

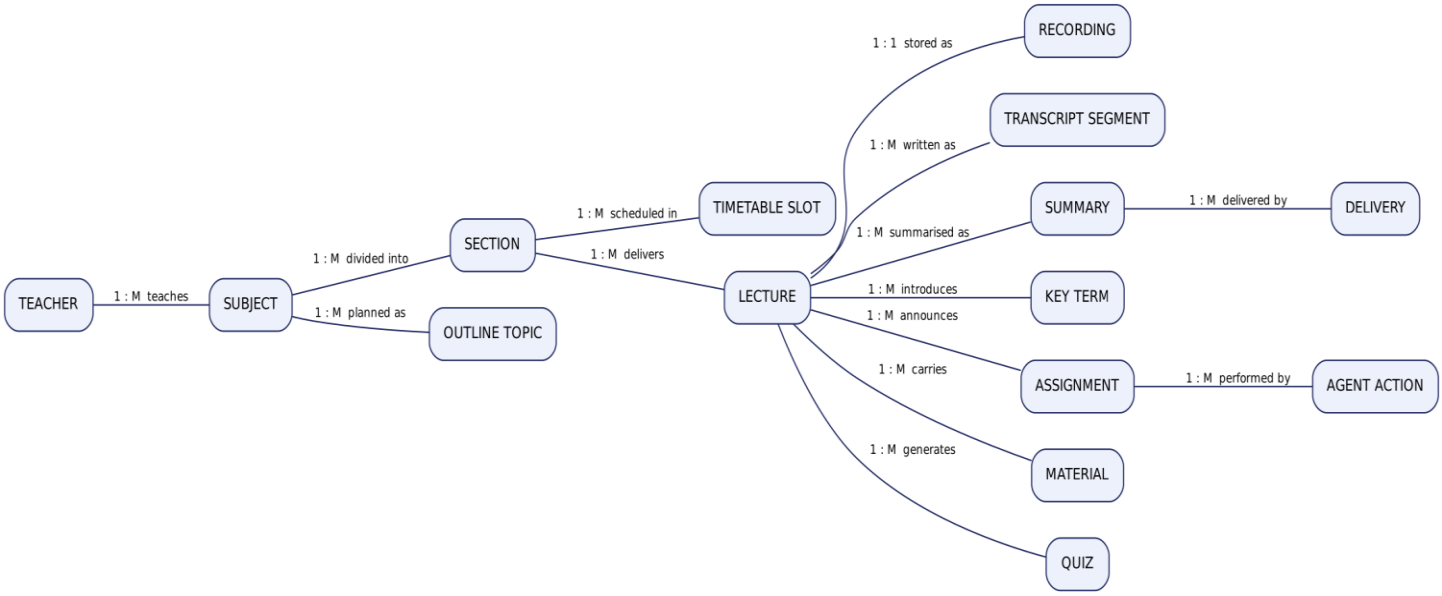



*Figure 7: Entity relationship diagram*

| Relationship | Cardinality | Meaning |
|---|---|---|
| Teacher teaches Subject | 1 : M | One teacher has many subjects; a subject in this system belongs to one teacher. |
| Subject is divided into Section | 1 : M | Two sections of the same subject keep separate lectures, numbers and calendars. |
| Section is scheduled in Timetable Slot | 1 : M | A section may meet more than once a week. |
| Subject is planned as Outline Topic | 1 : M | The outline is the list the delivered lectures are checked against. |
| Section delivers Lecture | 1 : M | Every lecture belongs to exactly one section. |
| Lecture is stored as Recording | 1 : 1 | One class is one recording, even when it was paused and resumed. |
| Lecture is written as Transcript Segment | 1 : M | Each segment carries its own start time, end time and speaker. |
| Lecture is summarised as Summary | 1 : M | The same lecture can have a short, a detailed and an exam focused summary. |
| Lecture introduces Key Term | 1 : M | The glossary of one class. |
| Lecture announces Assignment | 1 : M | Zero, one or several tasks may be announced in one class. |
| Assignment is performed by Agent Action | 1 : M | An upload may be attempted more than once; every attempt is recorded. |
| Summary is delivered by Delivery | 1 : M | The same summary may go to the class group and to the teacher's own chat. |
| Lecture generates Quiz | 1 : M | A quiz comes from one lecture; a paper comes from many lectures of a section. |

## *Sequence Diagram*

The sequence diagram below shows the main path of the system: the teacher presses record, and the result is a summary on the students' group and an assignment on the Moodle course page. It is the path that the first review will be judged on, and it is the one that every other feature is built around.

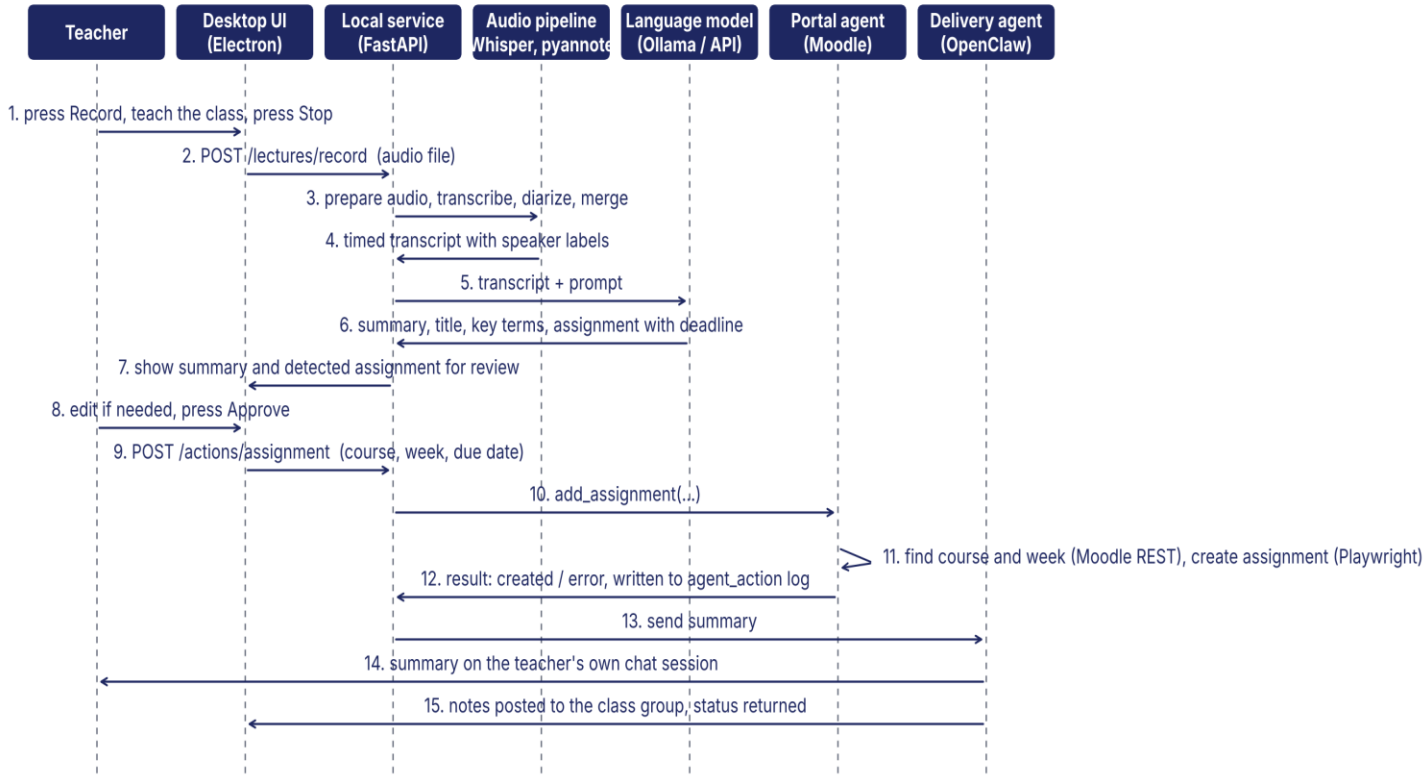



*Figure 8: Sequence of one lecture, from recording to portal upload and delivery*

Three points in this sequence matter for the design. Step 3 and step 4 are the only steps that touch the audio, and they run as a background job, so the interface stays usable while a fifty minute lecture is processed. Step 8 is the approval step: nothing before it leaves the machine, and nothing after it happens without the teacher. Step 12 writes the result of the outside action into the agent action log whether it succeeded or failed, which is what the log monitoring agent later reads.

## *System Interface Design*

The interface is designed around one rule taken from the requirements: a teacher has very little time, so the common path must be one button and the system must choose the subject and the section by itself. The three screens below cover the whole main path.

Home screen. The left side lists the subjects and sections built from the uploaded timetable. The right side shows the recording controls and the recent lectures of the selected section with their status, so the teacher can see at a glance which lectures are summarised, delivered or still waiting.

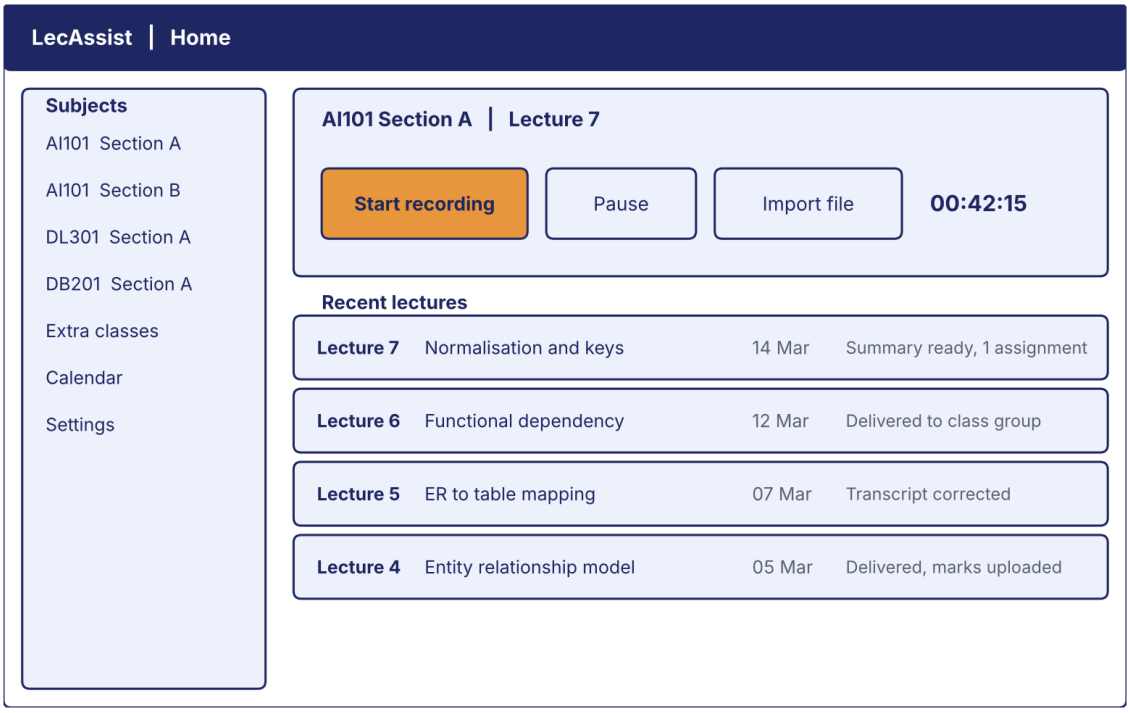



*Figure 9: Home screen with recording controls and the section archive*

Lecture screen. The transcript is on the left with the time, the speaker and the text of each line; clicking a line plays the audio from that second and a line can be corrected in place. The summary is on the right with the style selector, the key terms, the highlight and the detected assignment with its deadline. Every outside action is a button on this screen and each one opens the approval dialog first.

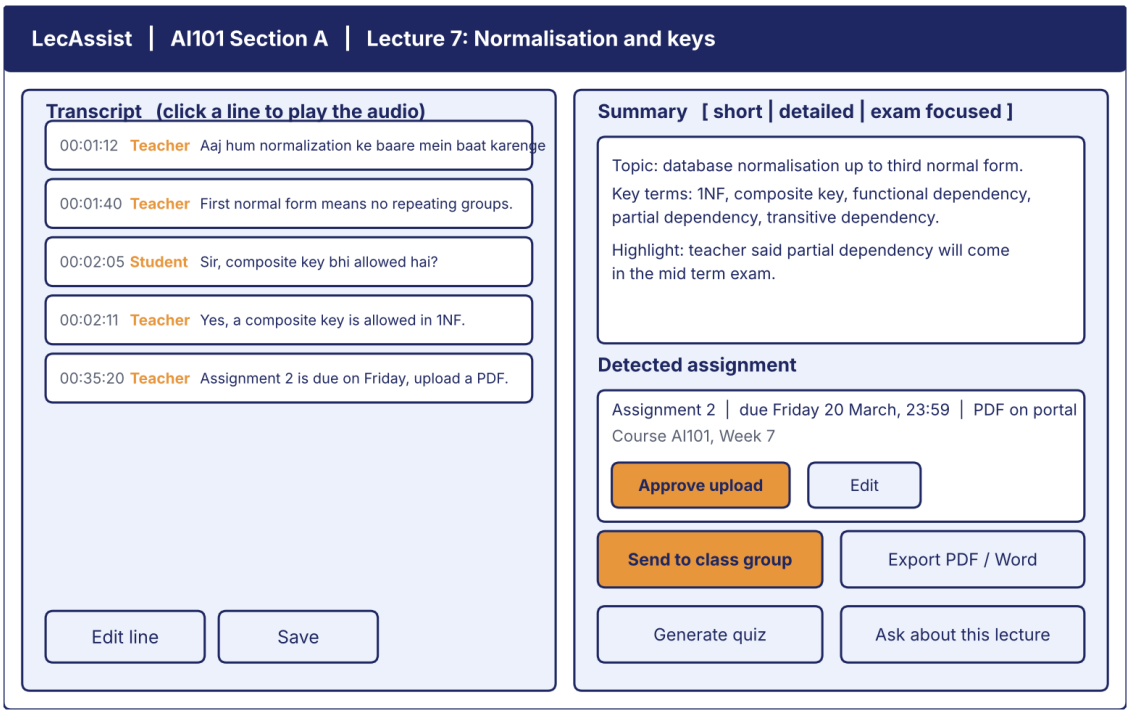



*Figure 10: Lecture screen with the transcript editor and the summary*

Approval and search screen. The approval dialog shows exactly what the agent is about to do, in plain words, with the course, the week, the name, the due date and the file. Under it the agent action log shows what has already been done and what failed. The right side is the semester search, where a question is answered from all the lectures of the section and every answer carries the lecture and date it came from.

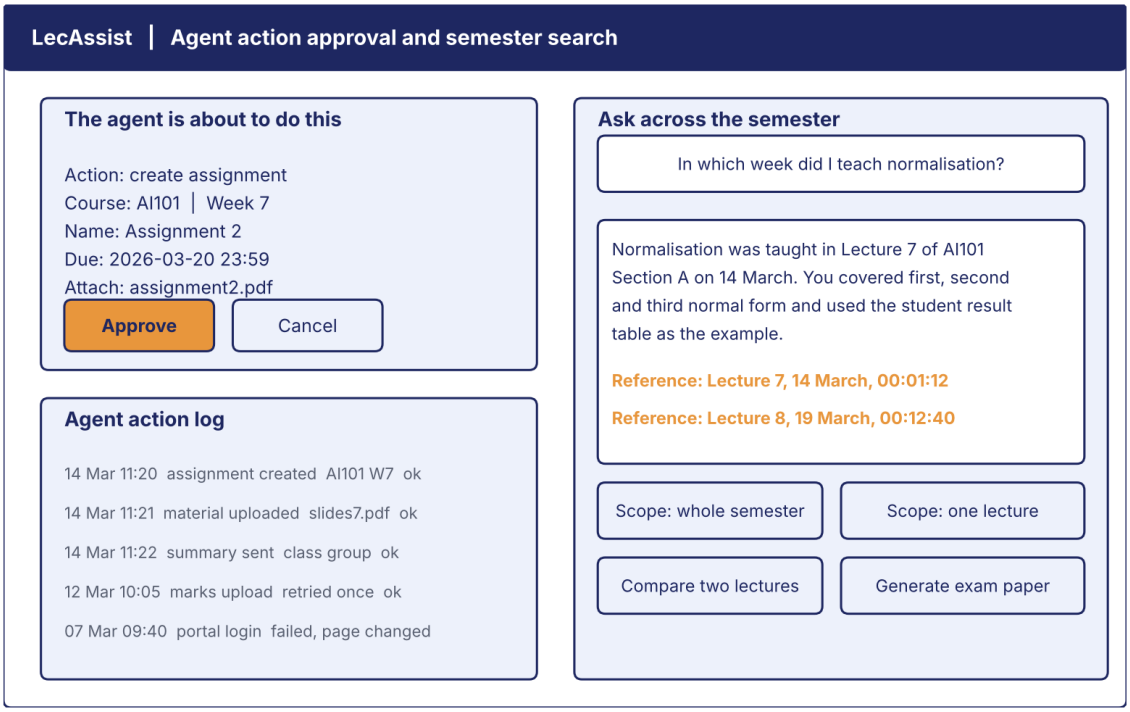



*Figure 11: Approval dialog, agent action log and semester search*

# Test Cases

The table below lists the test cases used to evaluate the system. They follow the order of the layers, because each layer is tested before the next one is built. The status column records whether the case already passes on the part of the system that is implemented.

| ID | Test case and steps | Expected result | Status |
|---|---|---|---|
| TC-01 | Convert input audio. Run the pipeline on an MP3 lecture clip recorded on a phone. | A 16 kHz mono WAV file is written to data/clean and the pipeline continues to the next stage. | Passing |
| TC-02 | Noise reduction. Run the pipeline on a noisy classroom clip with the denoise option on. | A separate denoised WAV is written and is used for both transcription and diarization. The original file is not changed. | Passing |
| TC-03 | Missing FFmpeg. Remove FFmpeg from PATH and run the pipeline. | The system reports that FFmpeg was not found and stops without creating a partial lecture record. | Passing |
| TC-04 | Local transcription. Transcribe a five minute mixed Urdu and English clip with the local model. | A JSON file of timed segments is produced, and sentences that change language halfway are written as spoken and not forced into one language. | Passing |
| TC-05 | Transcription accuracy. Run the pipeline with a hand written reference transcript of the same clip. | Word error rate, substitutions, deletions and insertions are reported. The mixed language parts are measured separately and the WER is at or below 25 percent. | In testing |
| TC-06 | Speaker separation. Run diarization on a clip in which a student asks a question. | Speaker turns are produced, each transcript line receives the speaker with the largest overlap, and the student question is stored under a different speaker label from the teacher. | Passing |
| TC-07 | Missing diarization token. Unset the Hugging Face token and run the pipeline. | The transcript is still produced, speakers are marked as unknown and the teacher is warned. The run does not crash. | Passing |
| TC-08 | Transcript correction. Edit a wrong technical term in one line and regenerate the summary. | The corrected line is stored permanently, is marked as edited, and the new summary and search results use the corrected text. | To be tested |
| TC-09 | Automatic filing. Finish a recording that starts inside a timetable slot of AI101 Section A. | The lecture is saved under AI101 Section A with the next lecture number, without the teacher choosing any folder. | To be tested |
| TC-10 | Unscheduled class. Finish a recording whose time matches no timetable slot. | The lecture is kept as an unscheduled class and the teacher is asked to mark it as an extra class, a revision class or to choose the subject. | To be tested |
| TC-11 | Summary styles. Generate the short, the detailed and the exam focused summary of one lecture. | Three different summaries are produced from the same transcript, with greetings and side talk removed, and a lecture title taken from the content. | In testing |
| TC-12 | Assignment extraction. Use a lecture in which the teacher says a report is due on Friday on the portal. | A structured assignment record is returned with the title, the resolved deadline date and the submission method, and the deadline appears on the subject calendar. | Passing |
| TC-13 | No assignment announced. Use a lecture in which nothing was announced. | The system reports that no assignment was found and creates no calendar entry. No task is invented. | In testing |
| TC-14 | Find a course and week. Ask the Moodle client for course AI101 and week 3. | The course id, short name and full name are returned, and the week section is found. An unknown short name returns a clear not found result instead of an error. | Passing |
| TC-15 | Create an assignment. Approve an assignment upload for a course and week on the test Moodle. | The agent logs in, creates the assignment with the due date, attaches the PDF, saves it, and the assignment is visible in that week of the course page. | Passing |
| TC-16 | Approval refused. Run the same upload and answer no at the approval step. | Nothing is sent to Moodle, the browser is never opened and the action is recorded as cancelled. | Passing |
| TC-17 | Portal failure. Change the login password and run an upload. | The agent stops, reports the login error, changes nothing on the portal and writes the failure to the agent action log. | Passing |

| | | | |
|---|---|---|---|
| TC-18 | Upload material. Ask the agent in plain words to upload a named slide deck to week 5. | The model calls the material tool with the correct course, week, display name and file path, and after approval the file appears on the course page. | Passing |
| TC-19 | Marks submission. Give the agent a prepared marks sheet for a quiz activity. | A preview of the rows is shown, approved rows are entered in the grade book, and unmatched student identifiers are reported and skipped without stopping the rest. | To be tested |
| TC-20 | Delivery. Send an approved summary to the class group and to the teacher's own chat. | The summary reaches both, the delivery status is stored, and a reply from the phone continues the same session. | To be tested |
| TC-21 | Delivery fallback. Break the WhatsApp channel and send a summary. | The system falls back to Telegram, delivers the summary and tells the teacher which channel was used. | To be tested |
| TC-22 | Semester question. Ask in which week normalisation was taught, with the whole semester as the scope. | The answer is produced from the stored lectures and carries the lecture number, the date and the time position of each source, and clicking a reference opens that line. | To be tested |
| TC-23 | Unanswerable question. Ask about a topic that was never taught in that subject. | The system says the answer is not in the stored lectures instead of inventing one. | To be tested |
| TC-24 | Quiz generation. Generate ten multiple choice questions from one lecture. | Every question and answer can be traced to something said in that class, and the quiz exports to PDF and Word. | To be tested |
| TC-25 | Coverage report. Upload a course outline and compare it with six delivered lectures. | The report lists the topics covered with their lecture numbers, the topics touched briefly and the topics still remaining. | To be tested |
| TC-26 | Offline run. Disconnect the machine from the internet and record, transcribe and summarise a lecture with the local provider. | The whole main path completes and no audio or text leaves the device. Only the portal and delivery steps are reported as unavailable. | In testing |
| TC-27 | Separate users. Log in as a second teacher on the same installation. | Only that teacher's subjects, lectures and settings are visible, and the first teacher's archive cannot be opened. | To be tested |
| TC-28 | Crash during recording. Kill the application in the middle of a recording. | The audio written so far is kept, the lecture is marked as unfinished, and processing resumes when the application is started again. | To be tested |
| TC-29 | Log monitoring. Run several failed and repeated agent actions, then run the monitoring agent. | The report lists the failed actions, the repeated actions, the unusual access and the steps that used too much time or too many tokens. | To be tested |
| TC-30 | End to end trial. Record a real class and follow the full path to delivery and upload. | A corrected transcript, a summary, a deadline on the calendar, an assignment on Moodle and notes on the class group are produced from one press of record, and the teacher's time on post lecture work is measured with and without the system. | To be tested |